

贪心与搜索

pwq

Cap1taL

2026 年 7 月

CodeForces 496E Distributing Parts

热身题

n 首歌曲，每首的音域为 $[a_i, b_i]$

m 个演员，每个人的音域范围为 $[c_i, d_i]$ ，他最多可以演唱 k_i 首歌曲

一首歌曲能被一个演员演唱，当且仅当演员 $[c, d]$ 完全包含歌曲 $[a, b]$

判断是否存在一个分配歌曲演唱的方案, 并给出构造

$$n, m \leq 10^5$$

其余不重要

贪心与搜索

将演员与歌曲，一同按区间左端点升序排列，从左向右依次考虑它们

将演员与歌曲，一同按区间左端点升序排列，从左向右依次考虑它们

- ① 若考虑到演员-左端，将其纳入维护
- ② 若考虑到演员-右端，将其移出维护
- ③ 若考虑到歌曲-左端，我们从维护的演员里，找到能演唱当前歌曲，且右端最小的演员，将其与当前乐曲匹配

现在考虑 k 的影响

我们将一个人可唱 k 次，等效为 k 个人，则可直接套用刚才的做法

我们将一个人可唱 k 次，等效为 k 个人，则可直接套用刚才的做法
 k 可以到 10^9 量级，因此我们不能真的拆人 (?!)

我们在纳入维护演员时，为每个人额外维护一个“剩余次数”即可

我们在纳入维护演员时，为每个人额外维护一个“剩余次数”即可
具体维护可以使用 set 或者 multiset

给定一个 $n \times m$ 的网格，每个格子是空地，或墙壁 *

房间定义为极大四连通空地连通块

可以拆除一些墙壁 (将 * 变成.), 目标是让每个房间都是矩形

求最少拆除数量，并输出任意一种方案

$$n, m \leq 2000$$

我们考虑：什么样的“房间”不是一个“矩形”？

1999, 2000, 2001, 2002, 2003, 2004, 2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025, 2026, 2027, 2028, 2029, 2030, 2031, 2032, 2033, 2034, 2035, 2036, 2037, 2038, 2039, 2040, 2041, 2042, 2043, 2044, 2045, 2046, 2047, 2048, 2049, 2050, 2051, 2052, 2053, 2054, 2055, 2056, 2057, 2058, 2059, 2060, 2061, 2062, 2063, 2064, 2065, 2066, 2067, 2068, 2069, 2070, 2071, 2072, 2073, 2074, 2075, 2076, 2077, 2078, 2079, 2080, 2081, 2082, 2083, 2084, 2085, 2086, 2087, 2088, 2089, 2090, 2091, 2092, 2093, 2094, 2095, 2096, 2097, 2098, 2099, 2100, 2101, 2102, 2103, 2104, 2105, 2106, 2107, 2108, 2109, 2110, 2111, 2112, 2113, 2114, 2115, 2116, 2117, 2118, 2119, 2120, 2121, 2122, 2123, 2124, 2125, 2126, 2127, 2128, 2129, 2130, 2131, 2132, 2133, 2134, 2135, 2136, 2137, 2138, 2139, 2140, 2141, 2142, 2143, 2144, 2145, 2146, 2147, 2148, 2149, 2150, 2151, 2152, 2153, 2154, 2155, 2156, 2157, 2158, 2159, 2160, 2161, 2162, 2163, 2164, 2165, 2166, 2167, 2168, 2169, 2170, 2171, 2172, 2173, 2174, 2175, 2176, 2177, 2178, 2179, 2180, 2181, 2182, 2183, 2184, 2185, 2186, 2187, 2188, 2189, 2190, 2191, 2192, 2193, 2194, 2195, 2196, 2197, 2198, 2199, 2200, 2201, 2202, 2203, 2204, 2205, 2206, 2207, 2208, 2209, 2210, 2211, 2212, 2213, 2214, 2215, 2216, 2217, 2218, 2219, 2220, 2221, 2222, 2223, 2224, 2225, 2226, 2227, 2228, 2229, 2230, 2231, 2232, 2233, 2234, 2235, 2236, 2237, 2238, 2239, 2240, 2241, 2242, 2243, 2244, 2245, 2246, 2247, 2248, 2249, 2250, 2251, 2252, 2253, 2254, 2255, 2256, 2257, 2258, 2259, 2260, 2261, 2262, 2263, 2264, 2265, 2266, 2267, 2268, 2269, 2270, 2271, 2272, 2273, 2274, 2275, 2276, 2277, 2278, 2279, 2280, 2281, 2282, 2283, 2284, 2285, 2286, 2287, 2288, 2289, 2290, 2291, 2292, 2293, 2294, 2295, 2296, 2297, 2298, 2299, 2300, 2301, 2302, 2303, 2304, 2305, 2306, 2307, 2308, 2309, 2310, 2311, 2312, 2313, 2314, 2315, 2316, 2317, 2318, 2319, 2320, 2321, 2322, 2323, 2324, 2325, 2326, 2327, 2328, 2329, 2330, 2331, 2332, 2333, 2334, 2335, 2336, 2337, 2338, 2339, 2340, 2341, 2342, 2343, 2344, 2345, 2346, 2347, 2348, 2349, 2350, 2351, 2352, 2353, 2354, 2355, 2356, 2357, 2358, 2359, 2360, 2361, 2362, 2363, 2364, 2365, 2366, 2367, 2368, 2369, 2370, 2371, 2372, 2373, 2374, 2375, 2376, 2377, 2378, 2379, 2380, 2381, 2382, 2383, 2384, 2385, 2386, 2387, 2388, 2389, 2390, 2391, 2392, 2393, 2394, 2395, 2396, 2397, 2398, 2399, 2400, 2401, 2402, 2403, 2404, 2405, 2406, 2407, 2408, 2409, 2410, 2411, 2412, 2413, 2414, 2415, 2416, 2417, 2418, 2419, 2420, 2421, 2422, 2423, 2424, 2425, 2426, 2427, 2428, 2429, 2430, 2431, 2432, 2433, 2434, 2435, 2436, 2437, 2438, 2439, 2440, 2441, 2442, 2443, 2444, 2445, 2446, 2447, 2448, 2449, 2450, 2451, 2452, 2453, 2454, 2455, 2456, 2457, 2458, 2459, 2460, 2461, 2462, 2463, 2464, 2465, 2466, 2467, 2468, 2469, 2470, 2471, 2472, 2473, 2474, 2475, 2476, 2477, 2478, 2479, 2480, 2481, 2482, 2483, 2484, 2485, 2486, 2487, 2488, 2489, 2490, 2491, 2492, 2493, 2494, 2495, 2496, 2497, 2498, 2499, 2500, 2501, 2502, 2503, 2504, 2505, 2506, 2507, 2508, 2509, 2510, 2511, 2512, 2513, 2514, 2515, 2516, 2517, 2518, 2519, 2520, 2521, 2522, 2523, 2524, 2525, 2526, 2527, 2528, 2529, 2530, 2531, 2532, 2533, 2534, 2535, 2536, 2537, 2538, 2539, 2540, 2541, 2542, 2543, 2544, 2545, 2546, 2547, 2548, 2549, 2550, 2551, 2552, 2553, 2554, 2555, 2556, 2557, 2558, 2559, 2560, 2561, 2562, 2563, 2564, 2565, 2566, 2567, 2568, 2569, 2570, 2571, 2572, 2573, 2574, 2575, 2576, 2577, 2578, 2579, 2580, 2581, 2582, 2583, 2584, 2585, 2586, 2587, 2588, 2589, 2590, 2591, 2592, 2593, 2594, 2595, 2596, 2597, 2598, 2599, 2600, 2601, 2602, 2603, 2604, 2605, 2606, 2607, 2608, 2609, 2610, 2611, 2612, 2613, 2614, 2615, 2616, 2617, 2618, 2619, 2620, 2621, 2622, 2623, 2624, 2625, 2626, 2627, 2628, 2629, 2630, 2631, 2632, 2633, 2634, 2635, 2636, 2637, 2638, 2639, 2640, 2641, 2642, 2643, 2644, 2645, 2646, 2647, 2648, 2649, 2650, 2651, 2652, 2653, 2654, 2655, 2656, 2657, 2658, 2659, 2660, 2661, 2662, 2663, 2664, 2665, 2666, 2667, 2668, 2669, 2670, 2671, 2672, 2673, 2674, 2675, 2676, 2677, 2678, 2679, 2680, 26

.#

我们考虑：什么样的“房间”不是一个“矩形”？

• •

.#

容易发现，既然房间不是矩形，必然存在上图所示的“拐角”（可以旋转）。且这样的拐角**必须被修复**

又发现，若我们修复了一个拐角，可能会引入新的拐角
搜索！我们抽象每个 2×2 的单元为一个新元素即可

$$n \leq 5 \times 10^5, \ a_i \leq 10^6$$

给出方案：

- 感性上容易理解，删除凹下去的部分是“不亏”的
- 设极小值 y 附近是 $[p, x, y, z, q]$ ，我们尝试证明：删除 x 或 z 之前先删除 y 一定不劣
 - A. 若先删除 x ，则收益为 $\min(p, y)$ ，剩余序列为 $[p, y, z, q]$
 - B. 若先删除 y ，则收益为 $\min(x, z)$ ，剩余序列为 $[p, x, z, q]$

“删除所有极小值点”

- 感性上容易理解，删除凹下去的部分是“不亏”的
- 设极小值 y 附近是 $[p, x, y, z, q]$ ，我们尝试证明：删除 x 或 z 之前先删除 y 一定不劣
- A. 若先删除 x ，则收益为 $\min(p, y)$ ，剩余序列为 $[p, y, z, q]$
- B. 若先删除 y ，则收益为 $\min(x, z)$ ，剩余序列为 $[p, x, z, q]$
- 注意到 B 方案的收益 $\min(x, z) \geq y \geq \min(p, y)$ ，且 B 的剩余序列更大，显然其剩余答案也大于 A
- 因此 B 方案更优，证明成立

- 显然最大值不可能进入答案
- 次大值成为答案，当且仅当“存在一次 $\min(\text{最大}, \text{次大})$ ”，而这在单峰序列中是不可能的

- 显然最大值不可能进入答案
- 次大值成为答案，当且仅当“存在一次 $\min(\text{最大}, \text{次大})$ ”，而这在单峰序列中是不可能的
- 因此答案必定不大于“除了最大值和次大值的元素之和”
- 构造方案：从“峰尖”开始取即可

- 11 / 40

N 头奶牛排队挤奶，每头牛需先后经过两道工序

若牛 i 先于牛 j 开始第一道工序, 则她也先开始第二道工序

求最优排队顺序下，完成所有挤奶的最短总时间

$$N \leq 25000, A_i, B_i \leq 2 \times 10^4$$

- A. 当 $a_i \leq b_i$ 时, 按照 a_i 升序
- B. 当 $a_i > b_i$ 时, 按照 b_i 降序
- 将 A 组放到 B 组的左侧

不乏感性理解方法。如何严格证明？

$$\max_{k=1}^n \left(\sum_{i=1}^k a_i + \sum_{i=k}^n b_i \right)$$

CapitalL 贪心与搜索

- 考虑何时交换 (a_i, b_i) 与 (a_j, b_j) 会变优
- 可得到 $\min(b_i, a_j) > \min(a_i, b_j)$

Solution

2. 临项交换

- 考虑何时交换 (a_i, b_i) 与 (a_j, b_j) 会变优
- 可得到 $\min(b_i, a_j) > \min(a_i, b_j)$
- 但它没有传递性。我们无法直接使用它排序。我们使用刚才构造的全序

Solution

2. 临项交换

- 考虑何时交换 (a_i, b_i) 与 (a_j, b_j) 会变优
- 可得到 $\min(b_i, a_j) > \min(a_i, b_j)$
- 但它没有传递性。我们无法直接使用它排序。我们使用刚才构造的全序
- 一方面，其满足我们推得的不等式；另一方面，任意满足我们推得不等式的排列，都可以通过交换变为全序排序后唯一的形式
- 证明？考虑交换一对元素对实际答案的影响

洛谷 P6631 序列

给定长度为 n 的非负整数序列 a

每一步可选择以下三种操作之一：

- 选择区间 $[l, r]$ ，所有数减 1
- 选择区间 $[l, r]$ ，其中下标为奇数的数减 1
- 选择区间 $[l, r]$ ，其中下标为偶数的数减 1

求将全部数归零的最少步数

$n \leq 10^5$, $a_i \leq 10^9$, $T \leq 10$

Solution

存在线性规划的套路做法，不过这不是我们今天的主题

Solution

存在线性规划的套路做法，不过这不是我们今天的主题

- 观察范围和时空限制，结合题面操作的高自由度，我们猜测是某种神秘贪心

Solution

存在线性规划的套路做法，不过这不是我们今天的主题

- 观察范围和时空限制，结合题面操作的高自由度，我们猜测是某种神秘贪心
- 注意到后两种操作其实是同一种，即隔一个操作一个

Solution

存在线性规划的套路做法，不过这不是我们今天的主题

- 观察范围和时空限制，结合题面操作的高自由度，我们猜测是某种神秘贪心
- 注意到后两种操作其实是同一种，即隔一个操作一个
- 一个常见的思考角度是从端点情况入手。我们考虑 a_n 和 a_{n-1}

Solution

存在线性规划的套路做法，不过这不是我们今天的主题

- 观察范围和时空限制，结合题面操作的高自由度，我们猜测是某种神秘贪心
- 注意到后两种操作其实是同一种，即隔一个操作一个
- 一个常见的思考角度是从端点情况入手。我们考虑 a_n 和 a_{n-1}
 - 若 $a_{n-1} = 0$ ，显然操作二不劣
 - 若 $a_n \neq 0$ ，不做任何事
 - 若 $a_{n-1}, a_n \neq 0$ ，虽然操作二可能延伸更远，但直接用操作一覆盖不劣。可以转化为上述情形
- 具体证明？

Solution

从右向左扫描，假设我们已经用刚才的方法把 $[L+1, n]$ 都变成 0，考虑 a_L

- 基本与刚才的情形类似，不同的是右侧“传来”了几个尚未结束的操作一/二或三。不妨假设其数量为 x 和 y 。

Solution

从右向左扫描，假设我们已经用刚才的方法把 $[L+1, n]$ 都变成 0，考虑 a_L

- 基本与刚才的情形类似，不同的是右侧“传来”了几个尚未结束的操作一/二或三。不妨假设其数量为 x 和 y 。
- 若 $a_L < x + y$ ，贪心选取，则必定有 $x + y - a_L$ 操作需要在 L 右侧结束
 - 我们暂时无法决策具体保留哪些操作，于是一个技巧是，将 x 与 y 均扣除 $x + y - a_L$ ，再“保留 $x + y - a_L$ 的自由操作额度”
 - x 和 y 可能会小于 $x + y - a_L$ ，需要先用鸽巢原理处理。

Solution

从右向左扫描，假设我们已经用刚才的方法把 $[L+1, n]$ 都变成 0，考虑 a_L

- 基本与刚才的情形类似，不同的是右侧“传来”了几个尚未结束的操作一/二或三。不妨假设其数量为 x 和 y 。
- 若 $a_L < x + y$ ，贪心选取，则必定有 $x + y - a_L$ 操作需要在 L 右侧结束
 - 我们暂时无法决策具体保留哪些操作，于是一个技巧是，将 x 与 y 均扣除 $x + y - a_L$ ，再“保留 $x + y - a_L$ 的自由操作额度”
 - x 和 y 可能会小于 $x + y - a_L$ ，需要先用鸽巢原理处理。
- 若 $a_L \geq x + y$ ，继续延伸右侧的操作——将 a_L 减去 $x + y$ 。此后若 a_L 仍不为 0，我们先尝试使用之前积累的“自由操作额度”，若不足再使用新操作
- 如何想到？

洛谷 P8314 Parkovi

给定一棵 n 个节点的树，边有长度
选择 k 个位置修建公园（同一节点最多一个）
最小化每个节点到最近公园距离的最大值
 $n \leq 2 \times 10^5$

Solution

- 一道小清新的树上贪心

Solution

- 一道小清新的树上贪心
- 最大值最小，首先套一个二分答案，问题转化为如何 check

Solution

- 一道小清新的树上贪心
- 最大值最小，首先套一个二分答案，问题转化为如何 check
- 类似上一道题！我们从端点（叶子）开始考虑

Solution

- 一道小清新的树上贪心
- 最大值最小，首先套一个二分答案，问题转化为如何 check
- 类似上一道题！我们从端点（叶子）开始考虑
- 使用贪心策略，则一个公园会被尽量放的更浅，以期望覆盖更多的节点

Solution

- 一道小清新的树上贪心
- 最大值最小，首先套一个二分答案，问题转化为如何 check
- 类似上一道题！我们从端点（叶子）开始考虑
- 使用贪心策略，则一个公园会被尽量放的更浅，以期望覆盖更多的节点
- 每个节点并不关心子树内的具体决策

Solution

- 一道小清新的树上贪心
- 最大值最小，首先套一个二分答案，问题转化为如何 check
- 类似上一道题！我们从端点（叶子）开始考虑
- 使用贪心策略，则一个公园会被尽量放的更浅，以期望覆盖更多的节点
- 每个节点并不关心子树内的具体决策
- 使用 DFS 遍历树，合并子树时先考虑能不能子树间相互覆盖，再考虑当前根节点是否必须放置公园
- 细节：根节点没有父节点

洛谷 P5665 划分

将数列 a 划分成若干连续段

要求每段之和**单调不降**

最小化每段之和的平方的和 (?! 绕绕?!)。即最小化：

$$\left(\sum_{i=1}^{k_1} a_i\right)^2 + \left(\sum_{i=k_1+1}^{k_2} a_i\right)^2 + \cdots + \left(\sum_{i=k_p+1}^n a_i\right)^2$$

$$n \leq 4 \times 10^7$$

Solution

- 题外话：感觉属于那种赛场上不严谨猜着做比严格证明简单的多的题

Solution

- 题外话：感觉属于那种赛场上不严谨猜着做比严格证明简单的多的题
- 首先有一个显然的 DP。设 $f_{i,j}$ 表示前 i 个数，最后一段是 $[j+1, i]$ 的最小答案。转移需要枚举倒数第二段的段首，复杂度 $O(n^3)$ 。

Solution

- 题外话：感觉属于那种赛场上不严谨猜着做比严格证明简单的多的题
- 首先有一个显然的 DP。设 $f_{i,j}$ 表示前 i 个数，最后一段是 $[j+1, i]$ 的最小答案。转移需要枚举倒数第二段的段首，复杂度 $O(n^3)$ 。
- 半感性地，我们知道 $(a+b)^2 = a^2 + b^2 + 2ab > a^2 + b^2$ 。即在保证合法的情况下，拆开一段总是更优的，每段更短更好。

Solution

- 题外话：感觉属于那种赛场上不严谨猜着做比严格证明简单的多的题
- 首先有一个显然的 DP。设 $f_{i,j}$ 表示前 i 个数，最后一段是 $[j+1, i]$ 的最小答案。转移需要枚举倒数第二段的段首，复杂度 $O(n^3)$ 。
- 半感性地，我们知道 $(a+b)^2 = a^2 + b^2 + 2ab > a^2 + b^2$ 。即在保证合法的情况下，拆开一段总是更优的，每段更短更好。
- 先给出结论：在所有合法的划分中，最优划分一定最后一段最短

Solution

“在所有合法的划分中，最优划分一定最后一段最短”

Solution

“在所有合法的划分中，最优划分一定最后一段最短”

- 数学归纳法。当 $n = 1$ 时显然成立，我们尝试证明 n 时也成立。

- 数学归纳法。当 $n=1$ 时显然成立，我们尝试证明 n 时也成立。
- 借用刚才 DP 转移的思想。
- 我们现在不需要再枚举 k ，数学归纳法保证了 f_j 的最优解的最后一段最短，则它的和也最小，最易合法。取更小的 k 不优。

- 数学归纳法。当 $n=1$ 时显然成立，我们尝试证明 n 时也成立。
- 借用刚才 DP 转移的思想。
- 我们现在不需要再枚举 k ，数学归纳法保证了 f_j 的最优解的最后一段最短，则它的和也最小，最易合法。取更小的 k 不优。
- 设 g_i 表示最优分拆下，最后一段的和。设 s_i 表示前缀和。转移条件可以写成 $g_j + s_j < s_i$ ，单调性表明可行的 j 是一个前缀。
- 问题转化为证明，取最大的 j 最优

“转移时取最大的 j 最优”

“转移时取最大的 j 最优”

- 我们证明任何一个 $k < j$ 都劣于 j

- 我们证明任何一个 $k < j$ 都劣于 j
- 为了简化问题，单独提出所有分拆点，可以证明 k 分拆的一段不可能被 j 分拆的一段完全包含。这表明排布一定形如先重合，后交替，最后一侧有一侧无的形式
- “最后一侧有一侧无”可以忽略

- 我们证明任何一个 $k < j$ 都劣于 j
- 为了简化问题，单独提出所有分拆点，可以证明 k 分拆的一段不可能被 j 分拆的一段完全包含。这表明排布一定形如先重合，后交替，最后一侧有一侧无的形式
- “最后一侧有一侧无”可以忽略
- 直接考虑二者各段的和序列为 $[X_1, X_n]$ 和 $[Y_1, Y_m]$ 。我们下面通过一系列操作，使 X_i 与 Y_i 相同，同时保证 X_i 不更优 Y_i 不更劣，从而说明原本的 X_i 优于 Y_i

Solution

“转移时取最大的 j 最优”

- 我们证明任何一个 $k < j$ 都劣于 j
- 为了简化问题，单独提出所有分拆点，可以证明 k 分拆的一段不可能被 j 分拆的一段完全包含。这表明排布一定形如先重合，后交替，最后一侧有一侧无的形式
- “最后一侧有一侧无”可以忽略
- 直接考虑二者各段的和序列为 $[X_1, X_n]$ 和 $[Y_1, Y_m]$ 。我们下面通过一系列操作，使 X_i 与 Y_i 相同，同时保证 X_i 不更优 Y_i 不更劣，从而说明原本的 X_i 优于 Y_i
- 找到最大的 p 最小的 q ，满足 $Y_1 = Y_p, Y_q = Y_m$ ，将 $Y_p + 1$ 同时 $Y_q - 1$ ，这使得 Y_i 变得更优
- 一旦 X_i 和 Y_i 存在了一段前缀相同，我们将前缀砍掉继续操作

- 最终 $Y_1 = Y_m$ 或 $Y_1 + 1 = Y_m$

Solution

“转移时取最大的 j 最优”

- 最终 $Y_1 = Y_m$ 或 $Y_1 + 1 = Y_m$
- 注意 $k < j$ 使得 $X_n \leq Y_m - 1 = Y_1$ ，由此得到 X_i 每个元素都小于 Y_1

- 最终 $Y_1 = Y_m$ 或 $Y_1 + 1 = Y_m$
- 注意 $k < j$ 使得 $X_n \leq Y_m - 1 = Y_1$, 由此得到 X_i 每个元素都小于 Y_1
 - 若两序列等长, 则二者必定完全恒等, 以此保证总和一致

Solution

“转移时取最大的 j 最优”

- 最终 $Y_1 = Y_m$ 或 $Y_1 + 1 = Y_m$
- 注意 $k < j$ 使得 $X_n \leq Y_m - 1 = Y_1$, 由此得到 X_i 每个元素都小于 Y_1
 - 若两序列等长, 则二者必定完全恒等, 以此保证总和一致
 - 若 $n = m + 1$, 我们把 X_1 分配给 $[X_2, X_n]$, 从而让二者相等, 同时 X_i 变的更劣

Solution

“转移时取最大的 j 最优”

- 最终 $Y_1 = Y_m$ 或 $Y_1 + 1 = Y_m$
- 注意 $k < j$ 使得 $X_n \leq Y_m - 1 = Y_1$, 由此得到 X_i 每个元素都小于 Y_1
 - 若两序列等长, 则二者必定完全恒等, 以此保证总和一致
 - 若 $n = m + 1$, 我们把 X_1 分配给 $[X_2, X_n]$, 从而让二者相等, 同时 X_i 变的更劣
- 综上, 我们总能通过一系列操作, 使 X_i 与 Y_i 相同, 同时保证 X_i 不更优, Y_i 不更劣

- 最终 $Y_1 = Y_m$ 或 $Y_1 + 1 = Y_m$
- 注意 $k < j$ 使得 $X_n \leq Y_m - 1 = Y_1$, 由此得到 X_i 每个元素都小于 Y_1
 - 若两序列等长, 则二者必定完全恒等, 以此保证总和一致
 - 若 $n = m + 1$, 我们把 X_1 分配给 $[X_2, X_n]$, 从而让二者相等, 同时 X_i 变的更劣
- 综上, 我们总能通过一系列操作, 使 X_i 与 Y_i 相同, 同时保证 X_i 不更优, Y_i 不更劣
- 因此原本的 X_i 优于 Y_i , 数学归纳法成立, 转移时只需要找到最大的合法 j 即可。

Solution

“转移时取最大的 j 最优”

- 最终 $Y_1 = Y_m$ 或 $Y_1 + 1 = Y_m$
- 注意 $k < j$ 使得 $X_n \leq Y_m - 1 = Y_1$, 由此得到 X_i 每个元素都小于 Y_1
 - 若两序列等长, 则二者必定完全恒等, 以此保证总和一致
 - 若 $n = m + 1$, 我们把 X_1 分配给 $[X_2, X_n]$, 从而让二者相等, 同时 X_i 变的更劣
- 综上, 我们总能通过一系列操作, 使 X_i 与 Y_i 相同, 同时保证 X_i 不更优, Y_i 不更劣
- 因此原本的 X_i 优于 Y_i , 数学归纳法成立, 转移时只需要找到最大的合法 j 即可。
- 使用数据结构可能会带 $\log$, 可以使用单调队列做到线性。卡常。

QOJ 5173 染色

一条纸条上有 n 个格子，第 i 个格子的颜色为 a_i
每秒钟可以执行以下两种操作之一：

- 将某个格子染成任意颜色
- 移动到相邻格子（要求移动前后颜色相同）

Q 次询问，每次求从格子 u 到 v 的最短时间
询问之间独立（不实际修改纸条）

$n, Q \leq 10^6$

Solution

- 先考虑单次询问怎么做，不妨设 $u < v$

Solution

- 先考虑单次询问怎么做，不妨设 $u < v$
- 最暴力的做法是每格都染，这给出了答案的上界

Solution

- 先考虑单次询问怎么做，不妨设 $u < v$
- 最暴力的做法是每格都染，这给出了答案的上界
- 发现若 u, v 之间存在一对 $i < j$ 满足 $a_i = a_j$ ，则我们可以少染一次
- 问题转化为怎么在 u, v 之间选出最多这样的区间

Solution

- 先考虑单次询问怎么做，不妨设 $u < v$
- 最暴力的做法是每格都染，这给出了答案的上界
- 发现若 u, v 之间存在一对 $i < j$ 满足 $a_i = a_j$ ，则我们可以少染一次
- 问题转化为怎么在 u, v 之间选出最多这样的区间
- 立马发现可以用简单的 DP 解决，并优化到线性
- 多次询问怎么办？

- 多次询问下发现 DP 没什么前途。这个问题这么简单，能不能不用 DP 呢？

- 多次询问下发现 DP 没什么前途。这个问题这么简单，能不能不用 DP 呢？
- 贪！心！预处理 R_i 表示 i 右侧最靠左的完整区间的右端点下标

- 多次询问下发现 DP 没什么前途。这个问题这么简单，能不能不用 DP 呢？
- 贪！心！预处理 R_i 表示 i 右侧最靠左的完整区间的右端点下标
- 我们断言：从 u 开始不断跳 R_i ，超过 v 之前跳的次数就是最大区间数
- 可以用反证法证明

- 多次询问下发现 DP 没什么前途。这个问题这么简单，能不能不用 DP 呢？
- 贪！心！预处理 R_i 表示 i 右侧最靠左的完整区间的右端点下标
- 我们断言：从 u 开始不断跳 R_i ，超过 v 之前跳的次数就是最大区间数
- 可以用反证法证明
- 具体实现方面，把询问挂到序列上，使用扫描线，同时使用并查集加树上差分，可以做到线性

QOJ 1173 Knowledge Is...

N 个项目, M 名学生

项目 i 在时间 L_i 可被领取, 必须在 R_i 或之前完成

一名学生不能同时做两个项目 (即若选 i, j , 须满足 $R_i < L_j$ 或 $R_j < L_i$)

每名学生最多做两个项目

求最多能完成的项目数, 并输出每个项目由哪名学生 (1~ M) 完成, 未完成输出 0

$N \leq 3 \times 10^5$

Solution

- 说人话：给一堆区间，选最多的匹配，满足匹配内的区间不交

Solution

- 说人话：给一堆区间，选最多的匹配，满足匹配内的区间不交
- 不太能 DP，尝试反悔贪心
- 套路地，我们将区间按右端点降序排序进行扫描

Solution

- 说人话：给一堆区间，选最多的匹配，满足匹配内的区间不交
- 不太能 DP，尝试反悔贪心
- 套路地，我们将区间按右端点降序排序进行扫描
 - 如果右侧存在区间能与当前区间匹配，我们直接进行匹配。我们尽量使用左端点更大的区间

Solution

- 说人话：给一堆区间，选最多的匹配，满足匹配内的区间不交
- 不太能 DP，尝试反悔贪心
- 套路地，我们将区间按右端点降序排序进行扫描
 - 如果右侧存在区间能与当前区间匹配，我们直接进行匹配。我们尽量使用左端点更大的区间
 - 如果不存在可以直接匹配的区间，我们尝试拆散已有匹配。设当前区间是 $[L, R]$ ，若存在匹配的最左端点大于 L ，我们将当前区间与匹配中的区间交换。我们尽量交换左端点更大的区间

Solution

- 说人话：给一堆区间，选最多的匹配，满足匹配内的区间不交
- 不太能 DP，尝试反悔贪心
- 套路地，我们将区间按右端点降序排序进行扫描
 - 如果右侧存在区间能与当前区间匹配，我们直接进行匹配。我们尽量使用左端点更大的区间
 - 如果不存在可以直接匹配的区间，我们尝试拆散已有匹配。设当前区间是 $[L, R]$ ，若存在匹配的最左端点大于 L ，我们将当前区间与匹配中的区间交换。我们尽量交换左端点更大的区间
 - 这样的交换总能扩大“未匹配区间”的待匹配范围，因此是更优的
- 使用优先队列维护即可

CodeForces 802M3 April Fools' Problem (hard)

有 n 天，需要准备 k 个问题

第 i 天最多准备一个问题（花费 a_i ），最多打印一个问题（花费 b_i ）

一个问题必须先准备再打印（可在同一天完成）

求准备并打印 k 个问题的最小总花费

$n \leq 5 \times 10^5$

Solution

- 原题要求必须打印 k 个问题

Solution

- 原题要求必须打印 k 个问题
- 容易发现点集 $\{(k, \text{mincost}_k)\}$ 是下凸的，使用 wqs 二分拿掉 k 的限制
 - 不会 wqs 二分的可以参考 P2619 [国家集训队] Tree I
 - wqs 二分并非今天的重点，现在可以先 skip 掉

Solution

- 原题要求必须打印 k 个问题
- 容易发现点集 $\{(k, \text{mincost}_k)\}$ 是下凸的，使用 wqs 二分拿掉 k 的限制
 - 不会 wqs 二分的可以参考 P2619 [国家集训队] Tree I
 - wqs 二分并非今天的重点，现在可以先 skip 掉
- 将 b_i 减去 wqs 二分的斜率后，变为纯粹的最优化问题

- 考虑一个简单的贪心：每天都尝试“打印”，如果前缀存在某天的“准备”开销和今天的“打印”开销加起来为负，就直接美美打印得吃

- 考虑一个简单的贪心：每天都尝试“打印”，如果前缀存在某天的“准备”开销和今天的“打印”开销加起来为负，就直接美美打印得吃
- 这显然是错误的，因为后面可能某天打印更划算

- 33 / 40

Solution

- 考虑一个简单的贪心：每天都尝试“打印”，如果前缀存在某天的“准备”开销和今天的“打印”开销加起来为负，就直接美美打印得吃
- 这显然是错误的，因为后面可能某天打印更划算
- 因此使用反悔贪心。打印的时候尝试能不能把之前的打印拆掉，比较一下与直接打印谁更优即可
- 具体实现可以用优先队列，把当前的 $-b_i$ 和正常的 a_i 混起来，之后如果取出

CodeForces 436E Cardboard Box

n 个关卡，每个关卡可以：

- 不玩（得 0 星）
- 花 a_i 时间通关得 1 星
- 花 b_i 时间通关得 2 星 ($a_i < b_i$)

每个关卡最多通关一次

至少要获得 w 颗星，求最短总时间

$$n \leq 3 \times 10^5$$

洛谷 P4511 日程管理

有 T 天，每天恰好可以完成一个任务

每个任务有截止日期 t_i 和收益 p_i ，必须在截止日前完成才获得收益

支持 Q 次操作：

- ADD t p ：添加一个任务
- DEL t p ：删除一个任务

每次操作后输出当前最大可获收益

$T, Q \leq 3 \times 10^5$

SPOJ MTREECOL Color a tree

给定一棵 N 个节点的树，根为 R

每个节点有染色代价因子 C_i

必须先染父节点才能染子节点

每染一个节点耗时 1，若节点 i 在时刻 F_i 完成染色，代价为 $C_i \times F_i$

求最小总染色代价

$N \leq 1000$, $C_i \leq 500$

洛谷 P3620 数据备份

N 栋办公楼在一条直线上，坐标 s_i 已排序
需要铺设 K 条电缆，每条连接两栋楼
每栋楼至多属于一对
最小化电缆总长度
 $N \leq 10^5$

CodeForces 2128F Strict Triangle

给定连通无向图 G 和节点 k

每条边 i 的权重 w_i 可在 $[l_i, r_i]$ 中任取实数

判断是否存在赋值方案，使得

$$\text{dist}_w(1, n) \neq \text{dist}_w(1, k) + \text{dist}_w(k, n)$$

多组数据， $\sum n, \sum m \leq 2 \times 10^5$

CodeForces 2034G2 Simurgh's Watch (Hard Version)

给定 n 个区间 $[l_i, r_i]$ ，每个区间需分配一种颜色

对任意整数时刻 t ，若至少有一个区间覆盖 t ，则存在一种颜色在 t 恰好只出现一次
求最小所需颜色数，并给出构造

$$\sum n \leq 2 \times 10^5$$

Thanks!